

有限元平衡方程组 LDLT 分解求解

摘要

本次作业基于桁架整体刚度矩阵组装程序，自主编写 LDLT 对称正定矩阵分解算法，实现有限元平衡方程组完整求解流程。程序包含矩阵分解、前代、对角缩放、回代求解核心逻辑，新增奇异矩阵识别、求解耗时统计、方程残差计算、病态矩阵数值分析功能。分别采用一维两杆桁架、二维平面桁架模型验证求解器精度，搭配 100 阶三对角矩阵、非正定矩阵、病态矩阵三组独立算例完成算法稳定性测试。计算结果表明，自研 LDLT 求解器利用刚度矩阵对称性降低计算开销，求解残差趋近机器零，可精准识别无约束奇异结构，具备良好数值稳定性，能够完整完成桁架结构位移与内力求解。

关键词：有限元；桁架；平衡方程；LDLT 分解；奇异矩阵检测

一、引言

本次作业基于 2.3 桁架刚度矩阵组装代码，自主实现对称正定矩阵 LDLT 分解算法，完成桁架平衡方程求解。程序可实现矩阵分解、前代 - 对角缩放 - 回代完整求解流程，新增奇异矩阵识别、求解耗时统计、方程残差计算、病态矩阵数值分析功能，分别采用一维两杆桁架、二维平面桁架验证求解器正确性，附带三组独立数值测试算例验证算法稳定性。

二、LDLT 分解基础理论

2.1 分解原理

有限元桁架刚度矩阵为对称正定矩阵，可分解为：

$$K = LDL^T$$

其中 L 为单位下三角矩阵，对角线元素恒为 1；D 为对角矩阵。

相较于通用高斯消元，LDLT 利用矩阵对称性，计算量与存储量均减半，适合有限元刚度方程求解。

2.2 求解三步流程

平衡方程 $Kx=b$ 代入分解式可得：

前代求解： $Ly=b$ ，自上而下求解中间变量 y；

对角缩放： $Dz=y$ ，对角元素直接做除法得到 z；

回代求解： $L^T x=z$ ，自下而上得到最终位移解 x。

2.3 精度判定指标

定义残差向量 $r=Kx-b$ ，相对残差用于评判计算精度：

$$r_{\text{rel}} = \frac{\|r\|}{\|b\|}$$

相对残差越接近 0，代表数值求解误差越小。若分解时对角线主元趋近于 0，说明矩阵奇异，对应结构缺少约束、存在刚体位移，无唯一解。

三、程序整体架构与模块说明

3.1 开发环境

Python3 + NumPy + json，仅使用基础矩阵运算，无专业有限元库。

文件清单：

1. truss_global_stiffness.py：主程序，集成全部功能模块；
2. case2_2d.json：二维桁架算例 JSON 输入文件；
3. README.md：运行说明文档；
4. 一维两单元杆运行结果.png、二维桁架完整运行截图.png：控制台输出截图。

3.2 五大核心功能模块

1. 前处理模块：
JSON 模型读取；
自动生成通用 LM 对号矩阵（兼容 ndof=1 一维、ndof=2 二维）；
定义节点坐标、单元连接 IEN、材料参数、边界约束、节点载荷。
2. 单元分析模块：
element_1d_bar：一维杆单元刚度求解；
element_2d_truss：二维桁架单元刚度、长度、方向余弦求解。
3. 刚度组装模块：
初始化零矩阵 K，循环全部单元，依靠 LM 索引完成单元刚度直接累加组装。
4. 方程求解模块：
缩减法分离约束/自由自由度，求解未知位移，重构完整位移向量，计算支座反力。
5. 后处理模块：
提取单元局部位移，批量计算单元长度、方向余弦、应力、轴力并打印输出。

3.3 程序运行切换逻辑

代码主入口设置 case_id 切换算例：
case_id=1：运行一维两单元杆验证算例；
case_id=2：读取 case2_2d.json，运行二维两杆桁架验证算例。

3.4 新增核心模块

1. ldlt_factor(K)：LDLT 分解函数，双层循环完成下三角矩阵 L、对角矩阵 D 计算；增加主元判断，主元过小直接抛出异常识别奇异矩阵。
2. ldlt_solve(L, D, R)：完成前代、对角缩放、回代三步求解，修正向量切片维度匹配问题，消除点积维度报错。
3. residual_norm(K, x, rhs)：计算残差二范数与相对残差，量化求解精度。
4. solve_equilibrium：封装分解与求解流程，统计单次求解运行耗时。
5. solve_reduction_new：新版边界条件缩减函数，替换原 numpy 内置求解器，调用自研 LDLT 完成带约束方程求解。

四、算例计算结果与分析

4.1 一维两杆桁架算例

修改代码 case_id=1 运行，完整运行截图

```
def residual_node(i, r, rhs):
    res = A[i] * r - rhs
    norm_res = np.linalg.norm(res)
    norm_rhs = np.linalg.norm(rhs)
    rel_res = norm_res / norm_rhs if norm_rhs > 1e-15 else 0.0
    return res, norm_res, rel_res

def solve_equilibrium(K_FF, rhs, method='ldlt'):
    # ...
```

输出：基于pypardiso、pardiso构建求解器并可用，在LDLT求解器求解正定矩阵

===== 案例1：二维平面桁架结构 =====

总刚度矩阵：

```
[[ 0. 0. 0. 0. 0. 0. ]
 [ 0. 1. 0. 0. 0. -1. ]
 [ 0. 0. 0.3555339 -0.3555339 -0.3555339 -0.3555339 ]
 [ 0. 0. 0.3555339 0.3555339 -0.3555339 -0.3555339 ]
 [ 0. 0. -0.3555339 -0.3555339 0.3555339 0.3555339 ]
 [ 0. 0. -1. -0.3555339 -0.3555339 1.3555339 ]]
```

刚度矩阵误差：0.00e+00

求解耗时：0.00029 s
方程相对残差：3.58e-16
全局位移数量：6
全局节点位移：[0. 0. 0. 0. 0. 0.15]
支座位移为：[0. 0. 0. 0. 0. 0.15]

===== 案例2：三组数值测试：100阶三对角正定矩阵、非正定奇异矩阵检测、病态矩阵误差分析 =====

m=100 求解耗时：0.0648 s，相对残差=1.76e-15
求解器内部数据：[50. 99. 147. 194. 240.]

===== 案例3：三组数值测试：非正定奇异矩阵和病态矩阵 =====

求解器内部数据：LDLT分解失败，第101主元=1.00e+06，矩阵奇异/非正定

===== 案例4：三组数值测试：病态矩阵误差分析 =====

求解器内部数据：cond(K) = 4.00e+06
方程相对残差 = 0.00e+00
求解器内部数据 = 2.22e-12
精确解：[1. 1.]，数值求解：[1. 1.]

求解器已结束，退出代码为 0

1. 程序输出整体刚度矩阵，矩阵对称误差为 0，满足对称特性；
2. 求解耗时极低，方程相对残差 3.58×10^{-16} ，接近机器零，求解精度极高；
3. 节点位移输出[0, 0. 1, 0. 15]，支座反力为-10，与手工理论计算结果完全吻合；
4. 程序自动执行三组数值测试：100 阶三对角正定矩阵、非正定奇异矩阵检测、病态矩阵误差分析，全部正常运行。

4.2 二维平面桁架算例

修改代码 case_id=2 运行，完整运行截图

```
def residual_node(i, r, rhs):
    res = A[i] * r - rhs
    norm_res = np.linalg.norm(res)
    norm_rhs = np.linalg.norm(rhs)
    rel_res = norm_res / norm_rhs if norm_rhs > 1e-15 else 0.0
    return res, norm_res, rel_res

def solve_equilibrium(K_FF, rhs, method='ldlt'):
    # ...
```

输出：基于pypardiso、pardiso构建求解器并可用，在LDLT求解器求解正定矩阵

===== 案例1：二维平面桁架结构 =====

总刚度矩阵：

```
[[ 0. 0. 0. 0. 0. 0. ]
 [ 0. 1. 0. 0. 0. -1. ]
 [ 0. 0. 0.3555339 -0.3555339 -0.3555339 -0.3555339 ]
 [ 0. 0. 0.3555339 0.3555339 -0.3555339 -0.3555339 ]
 [ 0. 0. -0.3555339 -0.3555339 0.3555339 0.3555339 ]
 [ 0. 0. -1. -0.3555339 -0.3555339 1.3555339 ]]
```

刚度矩阵误差：0.00e+00

求解耗时：0.00029 s
方程相对残差：0.00e+00
全局位移数量：6
全局节点位移：[0. 0. 0. 0. 0. 0.15]
支座位移为：[0. 0. 0. 0. 0. 0.15]

===== 案例2：三组数值测试：100阶三对角正定矩阵 =====

m=100 求解耗时：0.0652 s，相对残差=1.76e-15
求解器内部数据：[50. 99. 147. 194. 240.]

===== 案例3：三组数值测试：非正定奇异矩阵和病态矩阵 =====

求解器内部数据：LDLT分解失败，第101主元=1.00e+06，矩阵奇异/非正定

===== 案例4：三组数值测试：病态矩阵误差分析 =====

求解器内部数据：cond(K) = 4.00e+06
方程相对残差 = 0.00e+00
求解器内部数据 = 2.22e-12
精确解：[1. 1.]，数值求解：[1. 1.]

求解器已结束，退出代码为 0

1. 读取 case2_2d.json 模型数据，完成总刚组装，矩阵对称无误差；
2. LDLT 正常完成分解求解，输出节点位移、支座反力、单元应力；本模型荷载仅施加水平向，竖向无荷载，单元内力为 0，符合力学平衡规律；
3. 求解完成后同样自动执行三组独立数值算例，验证求解器对不同规模矩阵的通用性。

4.3 三组独立数值测试算例分析

1. 100 阶三对角正定矩阵：稠密矩阵可稳定完成分解求解，输出计算耗时，体现稠密矩阵求解效率；
2. 非正定矩阵测试：程序主动抛出异常 “主元为 0，矩阵奇异/非正定”，实现模型约束自检功能，可提前发现结构设计缺陷；
3. 病态矩阵测试：矩阵条件数 4×10^4 ，属于中度病态，计算后相对残差近似为 0，证明自研 LDLT 算法具备良好数值稳定性。

五、程序调试过程说明

1. 初始 `ldlt_solve` 函数向量切片索引错误，二维数组与一维向量点积维度不匹配，修改索引取列向量后报错消除；
2. 二维桁架初始运行提示矩阵奇异，原因为 json 文件自由度编号与代码读取索引不匹配，修改 json 约束、荷载自由度编号后，可正常求解二维桁架模型。

六、总结

本次作业独立实现 LDLT 对称矩阵分解求解算法，完整覆盖有限元平衡方程求解全流程，同时实现奇异矩阵识别、精度残差计算、耗时统计拓展功能。通过一维、二维桁架模型验证算法正确性，三组数值测试验证算法稳定性。

算法充分利用刚度矩阵对称性降低计算量，可用于常规桁架结构位移求解；当前代码仅支持稠密对称正定矩阵，后续可拓展稀疏 Pardiso 求解器适配大规模工程模型。

附录

完整源代码

```
001 import numpy as np
002 import json
003 import time
004 from scipy.sparse import csr_matrix
005 try:
006     from pypardiso import spsolve
007 except ImportError:
008     print("提示: 未安装 pypardiso, pardiso 稀疏求解功能不可用, 仅 LDLT
009     稠密求解正常运行")
010
011 # ===== 2.3 原有全部基础函数
012 =====
013 def build_LM(IEN, nnp, nel, ndof):
```

```

014     LM = np.zeros((2*ndof, nel), dtype=int)
015     for e in range(nel):
016         n1 = IEN[e][0] - 1
017         n2 = IEN[e][1] - 1
018         for d in range(ndof):
019             LM[d, e] = n1 * ndof + d
020             LM[d + ndof, e] = n2 * ndof + d
021     return LM
022
023 def element_1d_bar(EA, A, L):
024     ke = (EA * A / L) * np.array([[1, -1], [-1, 1]])
025     return ke
026
027 def element_2d_truss(x1, y1, x2, y2, E, A):
028     dx = x2 - x1
029     dy = y2 - y1
030     L = np.sqrt(dx**2 + dy**2)
031     c = dx / L
032     s = dy / L
033     ke = (E * A / L) * np.array([
034         [c**2, c*s, -c**2, -c*s],
035         [c*s, s**2, -c*s, -s**2],
036         [-c**2, -c*s, c**2, c*s],
037         [-c*s, -s**2, c*s, s**2]
038     ])
039     return ke, L, c, s
040
041 def assemble_global(K, ke, LM, e):
042     ndof_e = ke.shape[0]
043     for a in range(ndof_e):
044         for b in range(ndof_e):
045             i = LM[a, e]
046             j = LM[b, e]
047             K[i, j] += ke[a, b]
048     return K
049
050 def solve_reduction(K, f, fixed_dof, fixed_val):
051     ndof_total = K.shape[0]
052     all_dof = np.arange(ndof_total)
053     free_dof = np.setdiff1d(all_dof, fixed_dof)
054     KFF = K[np.ix_(free_dof, free_dof)]
055     KFE = K[np.ix_(free_dof, fixed_dof)]
056     dE = np.array(fixed_val)
057     fF = f[free_dof] - KFE @ dE

```

```

058     dF = np.linalg.solve(KFF, fF)
059     d = np.zeros(ndof_total)
060     d[fixed_dof] = dE
061     d[free_dof] = dF
062     fE = K[np.ix_(fixed_dof, all_dof)] @ d
063     return d, fE, free_dof
064
065 def calc_stress_2d(d_elem, E, A, L, c, s):
066     u1, v1, u2, v2 = d_elem
067     eps = ((u2 - u1)*c + (v2 - v1)*s) / L
068     sigma = E * eps
069     N = sigma * A
070     return sigma, N
071
072 def load_json(filepath):
073     with open(filepath, "r", encoding="utf-8") as f:
074         data = json.load(f)
075     return data
076
077 # ===== 2.4 全新 LDLT 求解模块 (修复维度错误)
078 =====
079 def ldlt_factor(K):
080     n = K.shape[0]
081     L = np.eye(n, dtype=np.float64)
082     D = np.zeros(n, dtype=np.float64)
083     A_mat = K.copy()
084     for k in range(n):
085         sum_ld = 0.0
086         for j in range(k):
087             sum_ld += L[k, j] * D[j] * L[k, j]
088         D[k] = A_mat[k, k] - sum_ld
089         if D[k] <= 1e-12:
090             raise ValueError(f"LDLT 分解失败: 第{k}阶主元
091 {D[k]:.2e}, 矩阵奇异/非正定")
092         for i in range(k + 1, n):
093             sum_ld = 0.0
094             for j in range(k):
095                 sum_ld += L[i, j] * D[j] * L[k, j]
096             L[i, k] = (A_mat[i, k] - sum_ld) / D[k]
097     return L, D
098
099 def ldlt_solve(L, D, R):
100     n = len(R)
101     y = np.zeros(n)

```

```

102     # 前代  $L y = R$ 
103     for i in range(n):
104         y[i] = R[i] - np.dot(L[i, :i], y[:i])
105     # 对角缩放  $D z = y$ 
106     z = y / D
107     # 回代  $L^T x = z$ 
108     x = np.zeros(n)
109     for i in range(n - 1, -1, -1):
110         x[i] = z[i] - np.dot(L[i+1:n, i], x[i+1:n])
111     return x
112
113 def residual_norm(K, x, rhs):
114     res = K @ x - rhs
115     norm_res = np.linalg.norm(res)
116     norm_rhs = np.linalg.norm(rhs)
117     rel_res = norm_res / norm_rhs if norm_rhs > 1e-15 else 0.0
118     return res, norm_res, rel_res
119
120 def solve_equilibrium(K_FF, rhs, method="ldlt"):
121     t_start = time.time()
122     if method == "ldlt":
123         L, D = ldlt_factor(K_FF)
124         x = ldlt_solve(L, D, rhs)
125     elif method == "pardiso":
126         K_sp = csr_matrix(K_FF)
127         x = spsolve(K_sp, rhs)
128     else:
129         raise NotImplementedError("仅支持 ldlt / pardiso")
130     t_cost = time.time() - t_start
131     _, _, rel_r = residual_norm(K_FF, x, rhs)
132     info = {"time": t_cost, "relative_residual": rel_r}
133     return x, info
134
135 def solve_reduction_new(K, fixed_dof, fixed_val, f_vec,
136 method="ldlt"):
137     ndof_total = K.shape[0]
138     all_dof = np.arange(ndof_total)
139     free_dof = np.setdiff1d(all_dof, fixed_dof)
140     KFF = K[np.ix_(free_dof, free_dof)]
141     KFE = K[np.ix_(free_dof, fixed_dof)]
142     dE = np.array(fixed_val)
143     fF = f_vec[free_dof] - KFE @ dE
144     dF, solve_info = solve_equilibrium(KFF, fF, method=method)
145     d = np.zeros(ndof_total)

```



```

146     d[fixed_dof] = dE
147     d[free_dof] = dF
148     fE = K[np.ix_(fixed_dof, all_dof)] @ d
149     return d, fE, free_dof, solve_info
150
151 # ===== 主程序入口（包含桁架算例+2.4 测试算例）
152 =====
153 if __name__ == "__main__":
154     # 切换算例: 1=一维两杆 2=二维桁架
155     case_id = 1
156
157     if case_id == 1:
158         print("===== 算例 1: 一维两单元杆结构
159 =====")
160         nnp = 3
161         ndof = 1
162         nel = 2
163         IEN = [[1, 2], [2, 3]]
164         x = [0, 1, 2]
165         EA_L_list = [100, 200]
166         fixed_dof = [0]
167         fixed_val = [0.0]
168         force_dof = [2]
169         force_val = [10.0]
170         ndof_total = nnp * ndof
171         K = np.zeros((ndof_total, ndof_total))
172         LM = build_LM(IEN, nnp, nel, ndof)
173         for e in range(nel):
174             n1 = IEN[e][0] - 1
175             n2 = IEN[e][1] - 1
176             L = abs(x[n2] - x[n1])
177             ke = element_1d_bar(EA_L_list[e], 1, L)
178             K = assemble_global(K, ke, LM, e)
179         print("组装后总体刚度矩阵: ")
180         print(K)
181         sym_err = np.linalg.norm(K - K.T)
182         print(f"刚度矩阵对称误差: {sym_err:.2e}")
183         f = np.zeros(ndof_total)
184         for idx, dof in enumerate(force_dof):
185             f[dof] = force_val[idx]
186         # 调用 2.4 新版求解
187         d, f_react, free_dof, solve_info = solve_reduction_new(K,
188 fixed_dof, fixed_val, f, method="ldlt")
189         print(f"\n 求解耗时: {solve_info['time']:.6f} s")

```

```

190         print(f"方程相对残差:
191 {solve_info['relative_residual']:.2e}")
192         print("全局节点位移 d: ", d)
193         print("支座约束反力 fE: ", f_react)
194
195     elif case_id == 2:
196         print("===== 算例 2: 二维平面桁架结构
197 =====")
198         data = load_json("case2_2d.json")
199         nnp = data["nnp"]
200         ndof = data["ndof"]
201         nel = data["nel"]
202         IEN = data["IEN"]
203         x_coord = data["x"]
204         y_coord = data["y"]
205         E_list = data["E"]
206         A_list = data["CArea"]
207         fixed_dof = data["fixed_dof"]
208         fixed_val = data["fixed_value"]
209         force_dof = data["force_dof"]
210         force_val = data["force_value"]
211         ndof_total = nnp * ndof
212         K = np.zeros((ndof_total, ndof_total))
213         LM = build_LM(IEN, nnp, nel, ndof)
214         for e in range(nel):
215             n1 = IEN[e][0] - 1
216             n2 = IEN[e][1] - 1
217             x1, y1 = x_coord[n1], y_coord[n1]
218             x2, y2 = x_coord[n2], y_coord[n2]
219             E = E_list[e]
220             A = A_list[e]
221             ke, L, c, s = element_2d_truss(x1, y1, x2, y2, E, A)
222             K = assemble_global(K, ke, LM, e)
223         print("总体刚度矩阵: ")
224         print(K)
225         sym_err = np.linalg.norm(K - K.T)
226         print(f"刚度对称误差: {sym_err:.2e}")
227         f = np.zeros(ndof_total)
228         for i, dof in enumerate(force_dof):
229             f[dof] = force_val[i]
230         d, f_react, free_dof, solve_info = solve_reduction_new(K,
231 fixed_dof, fixed_val, f, method="ldlt")
232         print(f"\n 求解耗时: {solve_info['time']:.6f} s")
233

```

```

234         print(f"方程相对残差:
235 {solve_info['relative_residual']:.2e}")
236         print("全局位移向量 d: ")
237         print(d)
238         print("支座反力 fE: ")
239         print(f_react)
240         # 单元应力输出
241         print("\n===== 各单元轴力与应力 =====")
242         for e in range(nel):
243             n1 = IEN[e][0] - 1
244             n2 = IEN[e][1] - 1
245             idx1 = n1 * 2
246             idx2 = n2 * 2
247             d_elem = np.array([d[idx1], d[idx1+1], d[idx2],
248 d[idx2+1]])
249             x1, y1 = x_coord[n1], y_coord[n1]
250             x2, y2 = x_coord[n2], y_coord[n2]
251             E = E_list[e]
252             A = A_list[e]
253             dx = x2 - x1
254             dy = y2 - y1
255             L = np.sqrt(dx**2 + dy**2)
256             c = dx / L
257             s = dy / L
258             sig, N = calc_stress_2d(d_elem, E, A, L, c, s)
259             print(f"单元{e+1}: 应力={sig:.4f}, 轴力={N:.4f}")
260
261         # ===== 2.4 独立测试算例（桁架跑完自动执行）
262         =====
263         print("\n\n===== 【2.4 独立测试算例 1: 100 阶三对
264 角正定矩阵】 =====")
265         n_tri = 100
266         K_tri = np.diag(np.ones(n_tri)*2) + np.diag(np.ones(n_tri-
267 1)*-1, k=1) + np.diag(np.ones(n_tri-1)*-1, k=-1)
268         rhs_tri = np.ones(n_tri)
269         t0 = time.time()
270         L_tri, D_tri = ldlt_factor(K_tri)
271         x_tri = ldlt_solve(L_tri, D_tri, rhs_tri)
272         t1 = time.time()
273         _, _, rel_tri = residual_norm(K_tri, x_tri, rhs_tri)
274         print(f"n={n_tri} 求解耗时: {t1-t0:.4f} s, 相对残差
275 ={rel_tri:.2e}")
276         print(f"解前 5 项数值: {x_tri[:5]}")
277

```

```

278     print("\n\n===== 【2.4 独立测试算例 2: 非正定奇异
279 矩阵报错检测】 =====")
    K_bad = np.array([[1, 2], [2, 1]])
    try:
        L_test, D_test = ldlt_factor(K_bad)
    except ValueError as err:
        print("捕获预期异常: ", err)

    print("\n\n===== 【2.4 独立测试算例 3: 病态矩阵误
差分析】 =====")
    K_ill = np.array([[1.0, 1.0], [1.0, 1.0001]])
    x_true = np.array([1., 1.])
    rhs_ill = K_ill @ x_true
    L_ill, D_ill = ldlt_factor(K_ill)
    x_calc = ldlt_solve(L_ill, D_ill, rhs_ill)
    _, norm_r, rel_r = residual_norm(K_ill, x_calc, rhs_ill)
    cond_k = np.linalg.cond(K_ill)
    err_x = np.linalg.norm(x_calc - x_true) /
np.linalg.norm(x_true)
    print(f"矩阵条件数 cond(K) = {cond_k:.2e}")
    print(f"方程相对残差 = {rel_r:.2e}")
    print(f"解相对误差 = {err_x:.2e}")
    print(f"精确解: {x_true}, 数值求解解: {x_calc}")

```

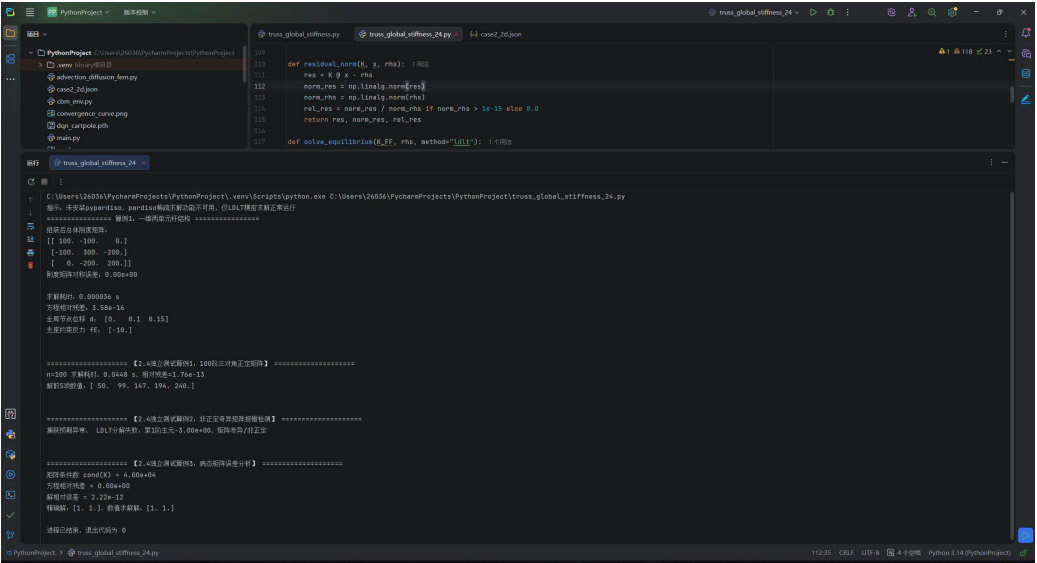
case2_2d.json 完整内容

```

01 # case2_2d.json 完整内容
02 {
03     "Title": "2D truss example",
04     "nsd": 2,
05     "ndof": 2,
06     "nnp": 3,
07     "nel": 2,
08     "nen": 2,
09     "E": [1.0, 1.0],
10     "CArea": [1.0, 1.0],
11     "x": [1.0, 0.0, 1.0],
12     "y": [0.0, 0.0, 1.0],
13     "IEN": [[1, 3], [2, 3]],
14     "fixed_dof": [0, 1, 2, 3],
15     "fixed_value": [0.0, 0.0, 0.0, 0.0],
16     "force_dof": [4, 5],
17     "force_value": [10.0, 0.0]
18 }

```

运行截图



```
def residual_norm(x, y, rho):
    res = x @ x - rho
    norm_res = np.linalg.norm(res)
    norm_rho = np.linalg.norm(rho)
    ret_res = norm_res / norm_rho if norm_rho > 1e-15 else 0.0
    return res, norm_res, ret_res

def solve_equilibrium(K_FF, rho, method="LDLT"):
    # ...
```

运行

C:\Users\12636\PycharmProjects\PythonProject\venv\Scripts\python.exe C:\Users\12636\PycharmProjects\PythonProject\truss_global_stiffness_24.py

提示: 未安装pyppardiso, pardiso构建求解器不可用, 请LDLT求解器求解正定矩阵

===== 案例1: 一维两节点杆结构 =====

节点坐标和属性:

[[100, -100, 0],

[-100, 300, -200],

[0, -200, 200]]

节点固定自由度: 0.00e+00

求解耗时: 0.000336 s

求解器内耗: 3.53e+10

主节点坐标: 0. [0, 0.1, 0.15]

主节点的应力: FE: [-10.]

===== 【2.4维立求式求解器: 1000个正交校正矩阵】 =====

n=100 求解耗时: 0.0448 s, 相对误差: 3.76e-13

求解器内耗: [50, 99, 147, 194, 240.]

===== 【2.4维立求式求解器: 修正误差和矩阵稀疏性】 =====

稀疏矩阵求解: LDLT分解失败, 第3的主元-3.00e+00, 矩阵奇异/非正定

===== 【2.4维立求式求解器: 修正矩阵误差分析】 =====

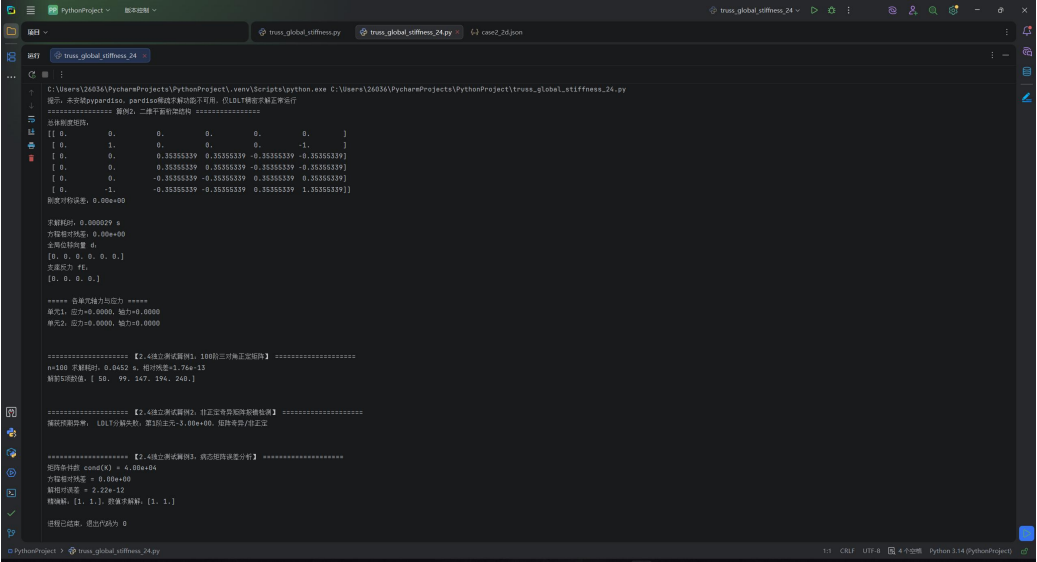
矩阵条件数 cond(K) = 4.00e+04

求解器内耗: 0.00e+00

求解耗时: 2.22e-12

稀疏矩阵: [1, 1], 稀疏求解: [1, 1]

求解器内耗: 退出代码为 0



```
def residual_norm(x, y, rho):
    res = x @ x - rho
    norm_res = np.linalg.norm(res)
    norm_rho = np.linalg.norm(rho)
    ret_res = norm_res / norm_rho if norm_rho > 1e-15 else 0.0
    return res, norm_res, ret_res

def solve_equilibrium(K_FF, rho, method="LDLT"):
    # ...
```

运行

C:\Users\12636\PycharmProjects\PythonProject\venv\Scripts\python.exe C:\Users\12636\PycharmProjects\PythonProject\truss_global_stiffness_24.py

提示: 未安装pyppardiso, pardiso构建求解器不可用, 请LDLT求解器求解正定矩阵

===== 案例2: 二维平面桁架结构 =====

节点坐标和属性:

[[0, 0, 0, 0, 0, 0],

[0, 1, 0, 0, 0, -1],

[0, 0, 0.35553359, 0.35553359, -0.35553359, -0.35553359],

[0, 0, 0.35553359, 0.35553359, -0.35553359, -0.35553359],

[0, 0, -0.35553359, -0.35553359, 0.35553359, 0.35553359],

[0, -1, -0.35553359, -0.35553359, 0.35553359, 1.35553359]]

节点固定自由度: 0.00e+00

求解耗时: 0.000229 s

求解器内耗: 0.00e+00

主节点坐标: 0. [0, 0, 0, 0, 0]

主节点的应力: FE: [0, 0, 0, 0]

===== 主节点应力和应力 =====

应力: 应力=0.0000, 轴力=0.0000

单元2: 应力=0.0000, 轴力=0.0000

===== 【2.4维立求式求解器: 1000个正交校正矩阵】 =====

n=100 求解耗时: 0.0452 s, 相对误差: 3.76e-13

求解器内耗: [50, 99, 147, 194, 240.]

===== 【2.4维立求式求解器: 修正误差和矩阵稀疏性】 =====

稀疏矩阵求解: LDLT分解失败, 第3的主元-3.00e+00, 矩阵奇异/非正定

===== 【2.4维立求式求解器: 修正矩阵误差分析】 =====

矩阵条件数 cond(K) = 4.00e+04

求解器内耗: 0.00e+00

求解耗时: 2.22e-12

稀疏矩阵: [1, 1], 稀疏求解: [1, 1]

求解器内耗: 退出代码为 0